

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/232623339>

A Model for Volume Lighting and Modeling

Article in IEEE Transactions on Visualization and Computer Graphics · May 2003

DOI: 10.1109/TVCG.2003.1196003 · Source: doi.ieeecomputersociety.org

CITATIONS

166

READS

267

5 authors, including:



Joe Kniss

University of New Mexico

47 PUBLICATIONS 3,305 CITATIONS

[SEE PROFILE](#)



Peter Shirley

NVIDIA

236 PUBLICATIONS 15,635 CITATIONS

[SEE PROFILE](#)



Allen McPherson

Los Alamos National Laboratory

19 PUBLICATIONS 382 CITATIONS

[SEE PROFILE](#)

Model for Volume Lighting and Modeling

Joe Kniss Simon Premože Charles Hansen Peter Shirley Allen McPherson

Abstract— Direct volume rendering is a commonly used technique in visualization applications. Many of these applications require sophisticated shading models to capture subtle lighting effects and characteristics of volumetric data and materials. For many volumes, homogeneous regions pose problems for typical gradient based surface shading. Many common objects and natural phenomena exhibit visual quality that cannot be captured using simple lighting models or cannot be solved at interactive rates using more sophisticated methods. We present a simple yet effective interactive shading model which captures volumetric light attenuation effects that incorporates volumetric shadows, an approximation to phase functions, an approximation to forward scattering, and chromatic attenuation that provides subtle appearance of translucency. We also present a technique for volume displacement or perturbation that allows realistic interactive modeling of high frequency detail for both real and synthetic volumetric data.

Keywords— Volume rendering, shading model, volume modeling, procedural modeling, fur, clouds, volume perturbation

I. INTRODUCTION

Direct volume rendering is widely used in visualization applications. Many of these applications render semi-transparent surfaces lit by an approximation to the Blinn-Phong local surface shading model. This shading model adequately renders such surfaces but it does not provide sufficient lighting characteristics for translucent materials or materials where scattering dominates the visual appearance. Furthermore, the normal required for the Blinn-Phong shading model is derived from the normalized gradient of the scalar field. While this normal is well defined for regions in the volume that have high gradient magnitudes, this normal is undefined in homogeneous regions, i.e. where the gradient is the zero vector, as seen in the left side of Figure 1. The use of the normalized gradient is also troublesome in regions with low gradient magnitudes, where noise can significantly degrade the gradient computation. It has been recently shown that volume rendering techniques can be used to directly visualize multi-variate datasets [15]. While a type of derivative measure can be computed for these datasets, it is not suitable for deriving a normal for surface shading. Shadows provide a robust mechanism for shading homogeneous regions in a volume and multi-variate field. They also substantially add to the visual perception of volume rendered data but shadows are not typically utilized with interactive direct volume rendering because of their high computational expense.

J. Kniss and C. Hansen are with the Scientific Computing and Imaging Institute, School of Computing, University of Utah. E-mail: jmk, hansen}@cs.utah.edu

S. Premože and P. Shirley are with the School of Computing, University of Utah. E-mail: premoze, shirley}@cs.utah.edu

A. McPherson is with the Advanced Computing Laboratory, Los Alamos National Laboratory. E-mail: mcpherson@lanl.gov

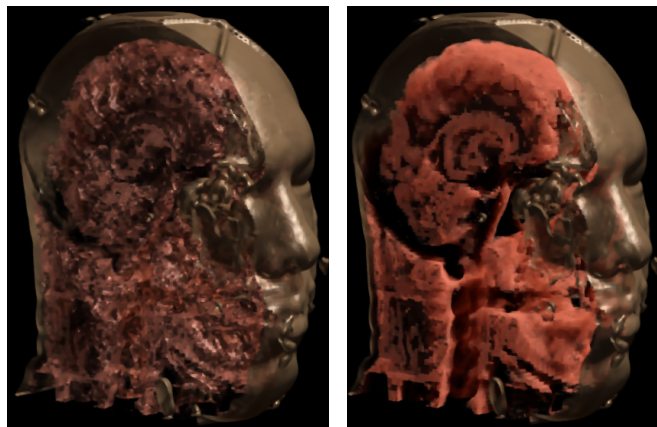


Fig. 1. Surface based shading versus volume shading model. Surface shading can not adequately shade homogeneous regions such as the soft tissue in a CT scan, as seen on the left. Our new, more general, volume shading model is needed to render the classified regions, as seen on the right.

Several studies have shown that the appearance of many common objects is dominated by scattering effects [4], [12]. This is especially true for natural phenomena such as smoke and clouds but it is also true for wax, skin, and other translucent materials.

While the effects of multiple scattering are important, physically accurate computation of them is not necessarily required for creating meaningful visualizations. In fact, interactive rendering for visualization already often employs such approaches (e.g. ambient light, OpenGL style fog, even the Blinn-Phong surface shading model). Interactivity for visualization is important since it aids in rapidly and accurately setting transfer functions [15], as well as providing important visual cues about spatial relationships in the data. While it is possible to precompute multiple scattering effects, for direct volume rendering such methods are dependent on the viewpoint, light position, transfer function and other rendering parameters which limits interactivity.

The ability to add detail to volumetric data has also been a computational bottleneck. Volume perturbation methods allow such effects at the cost of the time and memory to precompute such details before rendering. Volume perturbation methods have also been employed for modeling natural phenomena such as clouds. Such models, when coupled with a shading model with the visual appearance of scattering, can produce high quality visualizations of clouds and other natural phenomena, as well as introduce visually pleasing details to material boundaries (e.g. direct volume rendered isosurfaces) and statistically appropriate detail for low-resolution volume models.

In a previous paper, we presented a simple yet effective interactive volume shading model that captured effects

of volumetric light transport through translucent materials [16]. In this paper, we more thoroughly describe that shading model and how it relates to the classical volume shading. We also include forward peaked phase function into the model as well as other applications for volume perturbation. This achieves a method for interactive volumetric light transport that can produce effects such as direct lighting with phase angle influence, volumetric shadows, and a reasonable approximation of multiple scattering. This is shown in Figure 1. On the left is the standard surface shading of a CT scan of a head. On the right, rendered with the same transfer function, is our improved shading model. Leveraging the new light transport model allows for interactive volume modeling based on volumetric displacement or perturbation. This allows realistic interactive modeling of clouds, height fields, as well as the introduction of details to volumetric data.

In the next section, we introduce the problem of volume shading and light transport and describe related work on volume shading, scattering effects, as well as procedural modeling of clouds and surface detail. We then present the new volumetric shading model which phenomenologically mimics scattering and light attenuation through the volume. Implementation details are discussed and an interactive volumetric perturbation method is introduced. Results on a variety of volume data are also presented to demonstrate the effectiveness of these techniques.

II. BACKGROUND

In this section we review previous work and give an overview of volume shading equations.

A Previous Work

Volume visualization for scalar fields was described in three papers by Sabella, Drebin *et al* and Levoy [29], [7], [18]. These methods describe volume shading incorporating diffuse and specular shading by approximating the surface normal with the gradient of the 3D field. The volume rendering described in these seminal papers ignored scattering in favor of the fast approximation achieved by direct lighting. Techniques for implementing these approaches to volume rendering have been successfully implemented in hardware providing interactive volume rendering of 3D scalar fields [5], [25].

Blinn was one of the first computer graphics researchers to investigate volumetric scattering for computer graphics and visualization applications. He presented a model for the reflection and transmission of light through thin clouds of particles based on probabilistic arguments and single scattering approximation in which Fresnel effects were considered [3]. Kajiya and von Herzen described a model for rendering arbitrary volume densities that included expensive multiple scattering computation. The radiative transport equation [13] cannot be solved analytically except for some simple configurations. Expensive and sophisticated numerical methods must be employed to compute the radiance distribution to a desired accuracy. Finite element methods are commonly used to solve transport equa-

tions. Rushmeier presented zonal finite element methods for isotropic scattering in participating media [28], [27]. Max *et al.* [20] used a one-dimensional scattering equation to compute the light transport in tree canopies by solving a system of differential equations through the application of the Fourier transform. The method becomes expensive for forward peaked phase functions, as the hemisphere needs to be more finely discretized. Spherical harmonics were also used by Kajiya and von Herzen [14] to compute anisotropic scattering as well as discrete ordinate methods (Languenou *et al.* [17]).

Monte Carlo methods are robust and simple techniques for solving light transport equation. Hanrahan and Krueger modeled scattering in layered surfaces with linear transport theory and derived explicit formulas for backscattering and transmission [10]. The model is powerful and robust, but suffers from standard Monte Carlo problems such as slow convergence and noise. Pharr and Hanrahan described a mathematical framework [26] for solving the scattering equation in context of a variety of rendering problems and also described a numerical Monte Carlo sampling method. Jensen and Christensen described a two-pass approach to light transport in participating media [11] using a volumetric photon map. The method is simple, robust and efficient and it is able to handle arbitrary configurations. Dorsey *et al.* [6] described a method for full volumetric light transport inside stone structures using a volumetric photon map representation.

Stam and Fiume showed the the often used diffusion approximation can produce good results for scattering in dense media [31]. Recently, Jensen *et al.* introduced computationally efficient analytical diffusion approximation to multiple scattering [12], which is especially applicable for homogeneous materials that exhibit considerable subsurface light transport. The model does not appear to be easily extendible to volumes with arbitrary optical properties. Several other specialized approximations have been developed for particular natural phenomena. Nishita *et al.* [22] presented an approximation to light transport inside clouds and Nishita [21] an overview of light transport and scattering methods for natural environments [21]. These approximations are not generalizable for volume rendering applications because of the limiting assumptions made in deriving the approximations.

Max surveyed many optical models for volume rendering applications [19] ranging from very simple to very complex, and accurate models that account for all interactions within the volume.

Max [19] and Jensen *et al.* [12] clearly demonstrate that the effects of multiple scattering and indirect illumination are important for volume rendering applications. However, accurate simulations of full light transport are computationally expensive and do not permit interactivity such as changing the illumination or transfer function. Analytical approximations exist, but they are severely restricted by underlying assumptions, such as homogeneous optical properties and density, simple lighting or unrealistic boundary conditions. These analytical approximations cannot be

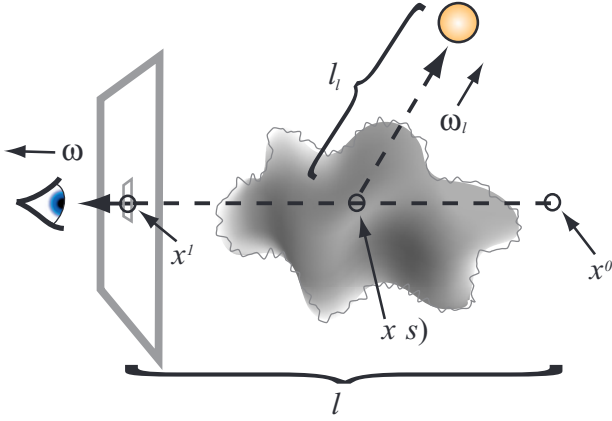


Fig. 2. Geometric setup used in volume shading equations.

Symbol	Definition
$\mathbf{x}$	Generic location
s	Distance from the ray's origin
$\mathbf{x}(s)$	Generic location along a ray
R	Surface reflectance
E	Emission in a volume
$T(s, l)$	Attenuation along the ray from $\mathbf{x}(s)$ to $\mathbf{x}(l)$
$\vec{\omega}$	Generic direction
$\vec{\omega}_l$	The light direction
τ	Attenuation coefficient
L_l	Point light source intensity
$L_l(s)$	Light intensity at point $\mathbf{x}(s)$
P	Phase function
l	Generic ray length
l_l	Light ray length

TABLE I

Important terms used in the paper

used for arbitrary volumes or real scanned data where optical properties such as absorption and scattering coefficients are hard to obtain.

B Volume Shading

The classic volume rendering model originally proposed by Levoy [18] deals with direct lighting only with no shadowing, with the basic geometry shown in Figure 2. If we parameterize a ray in terms of a distance from the background point $\mathbf{x}_0$ in direction $\vec{\omega}$ we have:

$$\mathbf{x}(s) = \mathbf{x}_0 + s\vec{\omega} \quad (1)$$

The classic volume rendering model is then written as:

$$L(\mathbf{x}_1, \vec{\omega}) = T(0, l)L(\mathbf{x}_0, \vec{\omega}) + \int_0^l T(s, l)R(\mathbf{x}(s))f_s(\mathbf{x}(s))L_l ds \quad (2)$$

where R is the surface reflectivity color, f_s is the Blinn-Phong surface shading model evaluated using the normal-

ized gradient of the scalar data field at $\mathbf{x}(s)$, and L_l is the intensity of a point light source. $L(\mathbf{x}_0, \vec{\omega})$ is the background intensity and T the amount the light is attenuated between two points in the volume:

$$T(s, l) = \exp \left(- \int_s^l \tau(s) ds \right) \quad (3)$$

and $\tau(s)$ is the attenuation coefficient at the sample s . This volume shading model assumes external illumination from a point light source that arrives at each sample unimpeded by the intervening volume. The only optical properties required for this model are an achromatic attenuation term and the surface reflectivity color, $R(\mathbf{x})$. Naturally, this model is well-suited for rendering surface-like structures in the volume, but performs poorly when attempting to render more translucent materials such as clouds and smoke. Often, the surface lighting terms are dropped and the surface reflectivity color, R , is replaced with the emission term, E :

$$L(\mathbf{x}_1, \vec{\omega}) = T(0, l)L(\mathbf{x}_0, \vec{\omega}) + \int_0^l T(s, l)E(\mathbf{x}(s))ds \quad (4)$$

This is often referred to as the emission/absorption model. As with the classical volume rendering model, the emission/absorption model only requires two optical properties, and E . In general, R , from the classical model, and E , from the emission/absorption model, are used interchangeably. This model also ignores inscattering. This means that although volume elements are emitting light in all directions, we only need to consider how this emitted light is attenuated on its path toward the eye.

Direct light attenuation, or volume shadows, can be added to the classical model as such:

$$L(\mathbf{x}_1, \vec{\omega}) = T(0, l)L(\mathbf{x}_0, \vec{\omega}) + \int_0^l T(s, l)R(\mathbf{x}(s))f_s(\mathbf{x}(s))T_l(s, l_l)L_l ds \quad (5)$$

where l_l is the distance from the current sample, $\mathbf{x}(s)$, to the light,

$$T_l(t, t) = \exp \left(- \int_0^t \tau(\mathbf{x}(t) + \vec{\omega}_l s) ds \right), \quad (6)$$

and $\vec{\omega}_l$ is the light direction. Just as T represents the attenuation from a sample to the eye, T_l is simply the attenuation from the sample to the light.

Phase function which accounts for directional distribution of light and indirect (inscattered) contributions are still missing in the volume shading model in equation 5. We will discuss the two missing terms in the following sections. Interested reader is referred to Arvo [1] and Max [19] for in depth derivations of volume light transport equations and volume shading models and approximations.

III. ALGORITHM

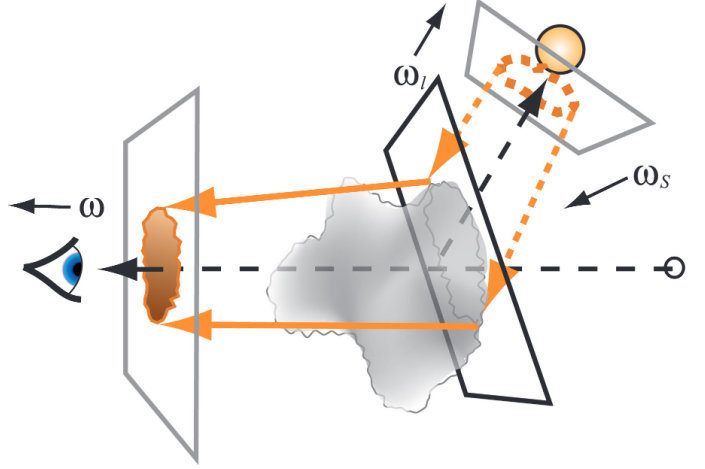
Our approximation of the transport equation is designed to provide interactive or near interactive frame rates for

volume rendering when the transfer function, light direction, or volume data are not static. Therefore, the light intensity at each sample must be recomputed every frame. Our method for computing light transport is done in image space resolutions, allowing the computational complexity to match the level of detail. Since the computation of light transport is decoupled from the resolution of the volume data, we can also accurately compute lighting for volumes with high frequency displacement effects, which are described in the second half of this section.

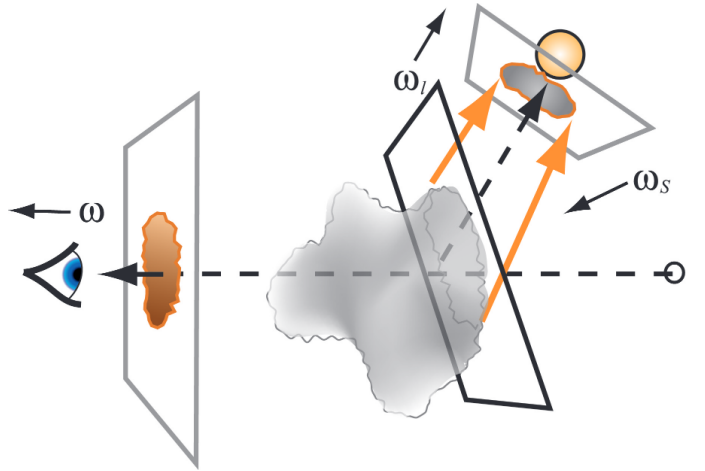
A Direct Lighting

Our implementation can be best understood if we first examine the implementation of direct lighting. A brute force implementation of direct lighting, or volumetric shadows, can be accomplished by sending a shadow ray toward the light for each sample along the viewing ray to estimate the amount of extinction caused by the portion of the volume between that sample and the light. This algorithm would have the computational complexity of $O(nm) \equiv O(n^2)$ where n is the total number of samples taken along each viewing ray, and m is the number of samples taken along each shadow ray. In general, the algorithm would be far too slow for interactive visualization. It is also very redundant since many of these shadow rays overlap. One possible solution would be to pre-compute lighting, by iteratively sampling the volume from the light's point of view, and storing the light intensities at each spatial position in a so called "shadow volume". While this approach reduces the computational complexity to $O(n+m) \equiv O(n)$, it has a few obvious disadvantages. First, this method can require a significant amount of additional memory for storing the shadow volume. When memory consumption and access times are a limiting factor, one must trade the resolution of the shadow volume, and thus the resolution of direct lighting computations, for reduced memory footprint and improved access times. Another disadvantage of shadow volume techniques is known as "attenuation leakage" this is caused by the interpolation kernel used when accessing the illumination in the shadow volume. If direct lighting could be computed in lock step with the accumulation of light for the eye, both integrals could be solved iteratively in image space using 2D buffers, one for storing light from the eye's point of view and another for the light source point of view. This can be accomplished using the method of half angle slicing proposed for volume shadow computation in [15], where the slice axis is halfway between the light and view directions or halfway between the light and inverted view directions depending on the sign of the dot product of the two. The modification of the slicing axis provides the ability to render each slice from the point of view of both the observer and the light. Thereby achieving the effect of a high resolution shadow map without the requirement of pre-computation and storage.

This can be implemented very efficiently on graphics hardware using texture based volume rendering techniques [33]. The approach requires an additional pass for each slice, which updates the the light intensities for the



(a) Step 1: Render for eye view, sample light buffer.



(b) Step 2: Update light buffer

Fig. 3. Two pass shadows using half angle slicing. For each slice, first render into the eye's buffer, sampling the positions that the slice projects to in the light buffer. Next render the slice into the light buffer, updating the light attenuation for the next slice. $\vec{\omega}_s$ indicates the slice axis.

next slice. The algorithm, by slicing the volume at intervals along some slice axis, $\vec{\omega}_s$, incrementally updating the color and opacity. Each slice is first rendered from the eye's point of view, where the light intensity at each sample on this slice is acquired by sampling the position it would project to in the light's buffer. This light intensity is multiplied by the color of the sample, which is then blended into the eye's buffer. This step is illustrated in Figure 3(a). Next this slice is rendered into the light buffer, attenuating the light by the opacity at each sample in the slice. This step is illustrated in Figure 3(b). An example of direct lighting in volume rendering applications can be seen in Figure 4.

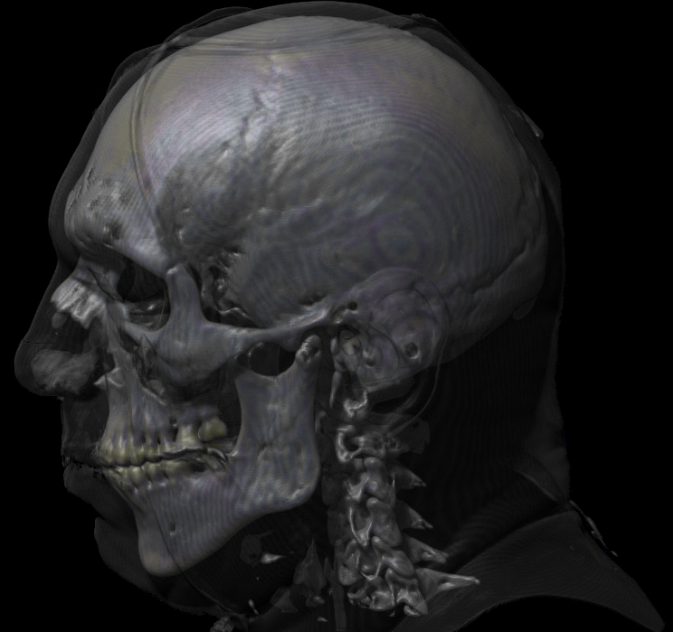


Fig. 4. An example of a volume rendering with direct lighting.

B Phase Functions

The role of the phase function in volume light transport is similar to that of the bidirectional reflectance distribution function (BRDF) in surface based light transport problems. It describes the distribution after a scattering event for each outgoing direction $\vec{\omega}$ given an incoming light direction $\vec{\omega}$. While the BRDF is only defined over a hemisphere of directions relative to the normal of the surface, the phase function describes the distribution of light over the entire sphere of directions. Phase function are only dependent on the cosine of the angle between the incoming and outgoing directions $\vec{\omega}$ and $\vec{\omega}$: $\cos\theta = \vec{\omega} \cdot \vec{\omega}$. While true phase function is normalized, $\int_4 P(\vec{\omega}, \vec{\omega}) d\omega = 1$, we leave our phase functions unnormalized. Figure 5 shows a plot of a phase function in polar coordinates. The radius r is essentially the weighting for a particular direction. Notice that the phase function is wavelength dependent, indicated by the colored contours. This class of phase functions is referred to as symmetric phase functions because the distribution of scattered energy is rotationally symmetric about the incoming direction. Symmetric phase functions are valid for spherical or randomly oriented particles. For most applications this class of phase functions is quite adequate.

Symmetrical phase functions can be implemented in conjunction with direct lighting by computing the dot product of the direction from the sample being rendered to the eye with the direction from the light to the sample, and then using this scalar value as an index into a one dimensional lookup table, i.e. this dot product is used as texture coordinate for reading from a 1D texture that stores the fraction of light scattered toward the eye. In our system, we compute these directions for each vertex that defines the corners of the current slice being rendered and apply them as texture coordinates. These coordinates are interpolated

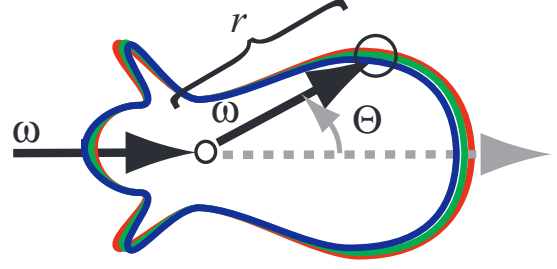


Fig. 5. An example of a symmetric phase function plotted in polar coordinates. The incoming direction $\vec{\omega}$ has fixed direction, while outgoing direction $\vec{\omega}$ varies over all directions.

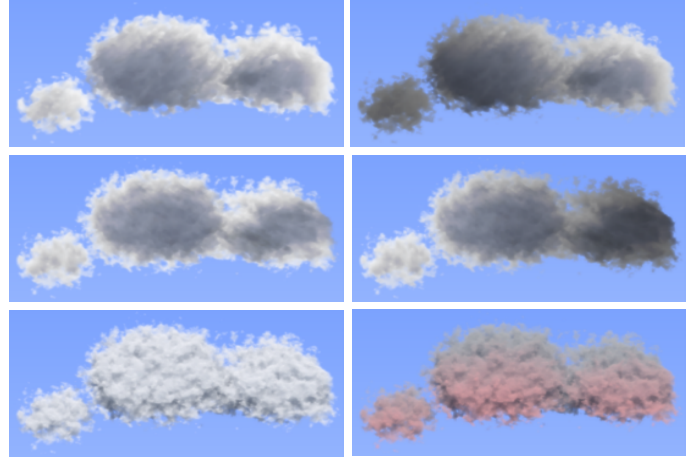


Fig. 6. Phase function effects. Images in the left column are rendered without the phase function modulating the direct light contribution while the images in the right column were rendered with the phase function. Each row has different incoming light direction.

over the slice during rasterization. In the per-fragment blending stage, we re-normalize these vectors and compute the dot product between them. Since the range of values resulting from the dot product are $[-1, 1]$, we first scale and bias the values, so that they are in the range $[0, 1]$, and then read from the 1D phase function texture. The result is then multiplied with direct lighting and reflective color. Figure 6 shows effects of the phase function.

C Indirect lighting approximation

Once direct lighting has been implemented in this way, computing the higher order scattering terms becomes a simple extension of this algorithm. As light is propagated from slice to slice, some scattering is allowed. This scattering is forward-only due to the incremental nature of the propagation algorithm. Thus this is an empirical approximation to the general light transport problem, and therefore its results must be evaluated empirically.

One major difference between our translucent volume shading model and traditional volume rendering approaches is the additional optical properties required for rendered to simulate higher order scattering. The key to understand our treatment of optical properties comes from recognizing the difference between absorption and attenu-

ation. The attenuation of light for direct lighting is proportional to extinction, which is the sum of absorption and outscattering.

The traditional volume rendering pipeline only requires two optical properties for each material: attenuation and material color. However, rather than specifying the attenuation term, which is a value in the range zero to infinity, a more intuitive opacity, or alpha, term is used:

$$= 1 - \exp(-\tau(\mathbf{x})). \quad (7)$$

The material color is the light emitted by the material in the simplified absorption/emission volume rendering model, however, the material color can be thought of as the diffuse reflectance if shadows or surface lighting are included in the model. In addition to these values, our model adds an indirect attenuation term to the transfer function. This term is chromatic, meaning that it describes the indirect attenuation of light for each of the R, G, and B color components. Similar to attenuation, the indirect attenuation can be specified in terms of an indirect alpha:

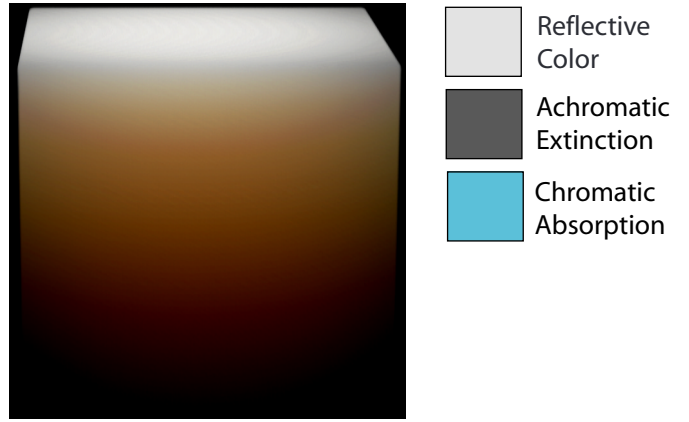
$$i = 1 - \exp(-\tau_i(\mathbf{x})) \quad (8)$$

While this is useful for computing the attenuation, we have found it non-intuitive for user specification. We prefer to specify a *transport color* which is $1 - i$ since this is the color the indirect light will become as it is attenuated by the material. Figure 7 illustrates the difference between the absorption, or indirect alpha, and the transport color. The alpha value can also be treated as a chromatic term (the details of this process can be found in [23]). For simplicity, we treat the alpha as an achromatic value since our aim is to clearly demonstrate indirect attenuation in interactive volume rendering.

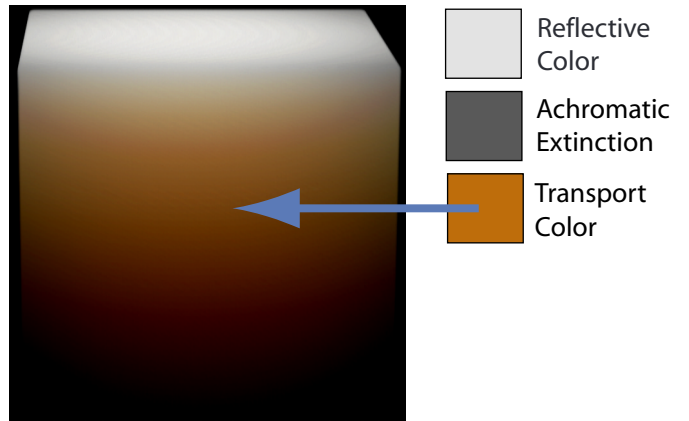
Our volume rendering pipeline computes the transport of light through the volume in lock step with the accumulation of light for the eye. Just as we update the direct lighting incrementally, the indirect lighting contributions are computed in the same way. Since we must integrate the incoming light over the cone of incoming directions, we need to sample the light buffer in multiple locations within this cone to accomplish the blurring of light as it propagates.

In the first pass, a slice is rendered from the observer's point of view. In this step, the transfer function is evaluated using a dependent texture read for the reflective color and alpha. In the hardware fragment shading stage, the reflective color is multiplied by the sum of one minus the indirect and direct light attenuation previously computed at that slice position in the **current** light buffer. This color is then blended into the observer buffer using the alpha value from the transfer function.

In the second pass, a slice is rendered into the **next** light buffer from the light's point of view to compute the lighting for the next iteration. Two light buffers are maintained to accommodate the blur operation required for the indirect attenuation. Rather than blend slices using a standard OpenGL blend operation, we explicitly compute the



(a) Optical properties using a chromatic absorption term.



(b) Optical properties using the transport color.

Fig. 7. To attenuate scattered light, we require chromatic indirect alpha, or absorption, term. However, this is very difficult for a user to specify. We prefer specifying the complement of this color, seen in (b), which is the color that light will become as it is attenuated.

blend in the fragment shading stage. The **current** light buffer is sampled once in the first pass, for the observer, and multiple times in the second pass, for the light, using the *render to texture* OpenGL extension. Whereas, the **next** light buffer, is rendered to only in the second pass. This relationship changes after the second pass so that the **next** buffer becomes the **current** and *vice versa*. We call this approach *ping pong blending*. In the fragment shading stage, the texture coordinates for the **current** light buffer, in all but one texture unit, are modified per-pixel using a random noise texture as discussed in the next section. The number of samples used for the computation of the indirect light is limited by the number of texture units. Currently, we use four samples. Randomizing the sample offsets masks some artifacts caused by this coarse sampling. The amount of this offset is restricted to a user defined blur angle (θ)